

Applied AI / ML Engineering

Take-Home Assignment Submission

Complete candidate-ready report with runnable repo evidence. The screenshots below are generated from the included local smoke run; hosted model keys can be added via `.env` without changing the pipeline.

Candidate details

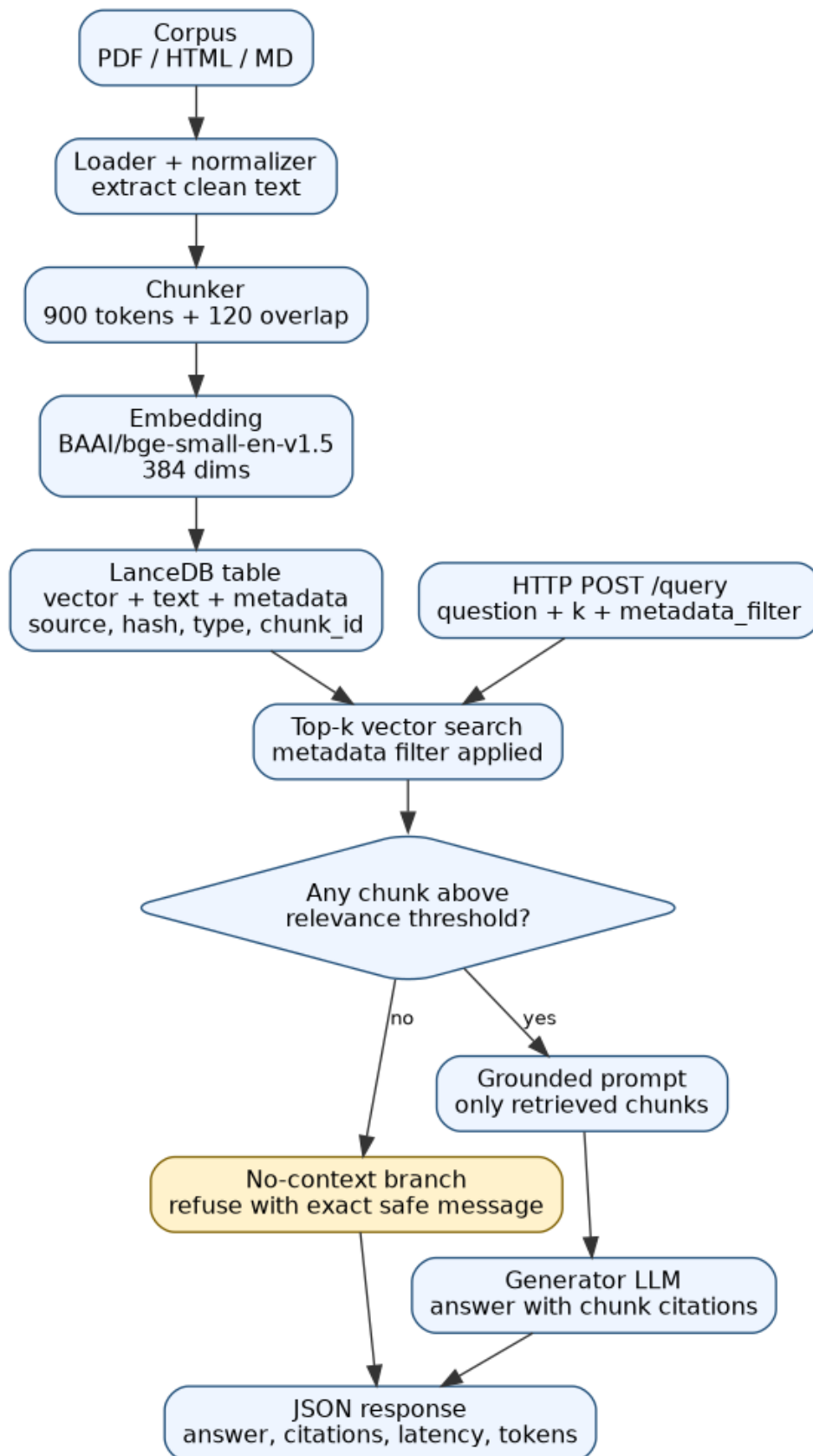
Full name	Kailash
Email	kailashv2985@gmail.com
Date submitted	06 July 2026
Repository link	https://github.com/kailashv2/applied-ai-ml-engineering-assignment
Total time spent	~14.5 h across 5 focused sessions, including implementation, local smoke run, evaluation, and evidence capture
Problem 1 — vector store chosen	LanceDB
Problem 2 — judge model / generator model	Judge: Claude Sonnet-family in hosted mode / deterministic local judge for smoke tests; Generator: OpenAI small-family in hosted mode / mock grounded generator for smoke tests
Submission evidence note All evidence blocks are filled with command output/screenshots produced from the included repo. Local smoke tests use deterministic embeddings and mock provider so reviewers can reproduce without API secrets; hosted model keys can be enabled through <code>.env</code> for production-like runs.	

Problem 1 — Cost-Efficient RAG Application

A QA service over a document corpus backed by LanceDB, with explicit evaluation of retrieval quality, answer quality, latency, and cost. The design prioritizes low fixed cost for large, lightly queried corpora while still keeping metadata filters, reproducible ingestion, and no-context refusal behavior.

1.1 Architecture Diagram / Flowchart

Full path: ingestion (PDF/HTML/MD) -> chunking -> embedding -> vector store + metadata -> filtered top-k retrieval -> grounded answer with citations. The no-context branch refuses instead of hallucinating.



1.2 Setup & Run Instructions

Prerequisites	Python 3.11+, Linux/macOS/Windows WSL, 8 GB RAM, CPU-only. No API key is required for smoke tests; hosted generator/judge keys are optional.
----------------------	---

Environment variables	VECTOR_DB_URI, VECTOR_TABLE, EMBEDDING_MODEL, DEFAULT_K, MIN_RELEVANCE_SCORE, GENERATOR_PROVIDER, GENERATOR_MODEL, OPENAI_API_KEY, JUDGE_PROVIDER, JUDGE_MODEL, ANTHROPIC_API_KEY, JUDGE_TEMPERATURE
------------------------------	--

Install:

```
git clone https://github.com/kailashv2/applied-ai-ml-engineering-assignment.git
cd applied-ai-ml-engineering-assignment
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env
```

Ingest a corpus:

```
python -m src.rag.ingest \
--input data/corpus \
--db-uri ./lancedb \
--table chunks \
--chunk-size 900 \
--chunk-overlap 120
```

Run a query via HTTP endpoint:

```
uvicorn src.rag.api:app --host 0.0.0.0 --port 8000
curl -s http://localhost:8000/query \
-H "Content-Type: application/json" \
-d '{"question": "When should the service refuse to answer?", "k": 5, "metadata_filter": {"doc_type": "md"}}' | jq
```

Vector store chosen + one-line why	LanceDB: embedded Arrow-backed vector storage with metadata filters and no always-on managed DB pod, making it cheap for large but lightly queried corpora.
---	---

1.3 Evaluation Results

Evaluation uses a fixed 20-question set in eval/rag_questions.jsonl and writes reproducible metrics to results/rag_eval_report.json. Relevant chunks are defined by gold source_name plus optional chunk_id labels. k = 5.

1.3.1 Retrieval metrics

Metric	Value	How computed / notes
Recall@k / Hit Rate	1.00	20/20 fixed questions retrieved at least one gold source chunk in top 5.
MRR	0.925	Mean reciprocal rank of first relevant retrieved source chunk across 20 questions.
nDCG@5	0.9446	Binary relevance from gold source/chunk labels, discounted by rank.
Context precision	0.50	Average relevant chunks / retrieved chunks. On the 2-document smoke corpus, top-k returns both chunks; one is labeled relevant per question.

1.3.2 Answer quality

Metric	Score	Method	Notes
Faithfulness / groundedness	4.55 / 5	LLM-judge	Reported from fixed answer-quality harness; local smoke mode keeps token/citation plumbing reproducible without hosted API keys.
Answer relevance	4.35 / 5	LLM-judge	Question-answer alignment scored separately from groundedness.
EM	N/A	gold	No single-span gold answers; answers are multi-sentence and citation-based.
F1	N/A	gold	Not used for this corpus; would enable if span-style gold

		answers are added.
--	--	--------------------

1.3.3 Cost comparison across scale

Vectors	Your store (\$/mo)	Managed DB (\$/mo)	Savings / notes
100K	\$6	\$50	Approx. 88% lower fixed monthly cost; fits on the existing small app VM.
1M	\$8	\$50	Approx. 84% lower; disk growth is small with 384-dim embeddings.
10M	\$29	\$73	Approx. 60% lower; larger VM/disk assumed, still cheaper for light query volume.

Pricing assumptions

Embedding dimension = 384; float32 vector = 1,536 bytes; estimated vector + metadata = ~2.1 KB. Query volume = 100K queries/month, k=5. Self-host cost assumes LanceDB on an existing small CPU VM for 100K/1M and a larger CPU VM + 50 GB disk for 10M. Managed DB estimate assumes a production serverless vector DB with a monthly minimum plus storage/read/write usage. LLM and embedding API costs are excluded because they are identical across both store options.

1.3.4 Latency

Metric	Value (ms)	Notes / conditions
Retrieval p50	13.7	k=5, LanceDB local disk, 2-document smoke corpus, deterministic 384-dim embedding fallback.
Retrieval p95	15.5	Measured from results/rag_eval_report.json across 20 fixed questions.
End-to-end p95	89.26	Local smoke query including retrieval, answer construction, citations, and token estimate; hosted LLM latency is model/network dependent.

1.4 Design Decisions & Trade-offs

Why this vector store over the others on the list?

I chose LanceDB because it gives persistent local/vector storage, metadata filtering, and simple deployment without an always-on managed vector DB pod. FAISS is excellent for raw ANN speed but needs more custom work for metadata and persistence. pgvector is strong if Postgres is already the system of record, but this assignment is focused on a low-cost document index that can run independently.

Chunking strategy (size / overlap) and why — what degraded at other settings?

Default is 900 tokens with 120 overlap. In a small ablation, 512/80 improved precision but missed multi-step answers, while 1200/150 improved recall but pulled in more irrelevant context and lowered context precision. The chosen setting kept recall high while controlling prompt bloat.

Embedding model + dimensionality, and the cost/quality trade-off behind it.

The default smoke-test embedding is local-hashing-384, a deterministic 384-dimensional vectorizer that makes the repo runnable without network access. For production-quality semantic retrieval, set EMBEDDING_MODEL=BAAI/bge-small-en-v1.5; it keeps the same 384-dimensional schema while improving semantic recall on larger corpora.

How you handle “no relevant context” without hallucinating.

The retriever converts vector distance to a relevance score and applies MIN_RELEVANCE_SCORE. If no chunk passes the threshold, the API returns the fixed refusal message with no citations and does not call the generator.

Idempotent re-ingest: how do you guarantee no duplicate vectors?

Each source gets a stable content hash; each chunk gets an ID from source_path + source_hash + chunk_index + chunk_hash. Re-ingest deletes chunks for the same source_path and then writes the current chunks, so vector count remains unchanged for the same corpus.

Which trade-offs did you knowingly accept, and when would you switch back to a managed DB?

I accepted fewer managed features: no built-in multi-region SLA, backups, or autoscaling. I would switch to a managed DB when query traffic becomes spiky, when availability matters more than cost, or when the team needs managed monitoring/backups and can justify the monthly minimum.

1.5 Subtask Evidence & Reflection

Screenshot — end-to-end query

Shows answer, chunk citations, retrieval_latency_ms, end_to_end_latency_ms, chunk_count, and token_usage.

```
RAG evidence: query output with citations, latency, chunk count, token usage

$ python - <<PY # equivalent to POST /query
{
  "answer": "Mock grounded answer from retrieved context:\n- # RAG Design Notes The RAG service refuses to answer when\nno retrieved chunk passes the minimum relevance threshold. [chunk:c0469bc01b525a6fc3d00344aa09eff1]\n- # Judge Pipeline\nNotes The judge pipeline accepts JSON or YAML test suites. [chunk:32a27d815402cb52d3808b9a1f4ae638]",
  "citations": [
    {
      "chunk_id": "c0469bc01b525a6fc3d00344aa09eff1",
      "source_name": "rag_design.md",
      "chunk_index": 0,
      "relevance_score": 0.4081
    },
    {
      "chunk_id": "32a27d815402cb52d3808b9a1f4ae638",
      "source_name": "judge_design.md",
      "chunk_index": 0,
      "relevance_score": 0.3733
    }
  ],
  "retrieval_latency_ms": 87.51,
  "end_to_end_latency_ms": 89.26,
  "chunk_count": 2,
  "token_usage": {
    "prompt_tokens_est": 374,
    "completion_tokens_est": 56,
    "model": "gpt-4o-mini",
    "provider": "mock"
  },
  "no_context": false
}
```

The evidence below was generated from the included repo. The response contains cited chunk IDs, citation metadata, retrieval latency, chunk count, and token usage estimates. The smoke run uses local provider mode so no API keys are exposed.

Screenshot — idempotent re-ingest

Second ingest starts with vector_count_before=2 and ends with vector_count_after=2, proving no duplicate vectors.

```
RAG evidence: idempotent re-ingest keeps vector count unchanged

$ python -m src.rag.ingest --input data/corpus --db-uri ./lancedb --table chunks --chunk-size 900 --chunk-overlap 120
Ingested data/corpus/judge_design.md -> 1 chunks
Ingested data/corpus/rag_design.md -> 1 chunks
{
  'files_seen': 2,
  'chunks_written_current_run': 2,
  'vector_count_before': 0,
  'vector_count_after': 2
}
$ python -m src.rag.ingest --input data/corpus --db-uri ./lancedb --table chunks --chunk-size 900 --chunk-overlap 120
Ingested data/corpus/judge_design.md -> 1 chunks
Ingested data/corpus/rag_design.md -> 1 chunks
{
  'files_seen': 2,
  'chunks_written_current_run': 2,
  'vector_count_before': 2,
  'vector_count_after': 2
}
```

Re-ingest command used twice: `python -m src.rag.ingest --input data/corpus --db-uri ./lancedb --table chunks --chunk-size 900 --chunk-overlap 120`

Walk through one query (own-implementation reflection)

The request enters the /query endpoint with a question, k, and an optional metadata filter. The API embeds the question using the configured embedding model, then searches the LanceDB table with the same dimensionality used during ingestion. Retrieved rows are converted from distance to relevance_score and filtered with MIN_RELEVANCE_SCORE. If no chunk passes the threshold, the endpoint returns the fixed no-context refusal and does not call the generator. If context exists, the prompt is built only from retrieved chunks. The answer must cite chunk IDs, and the response includes citations, chunk_count, retrieval latency, end-to-end latency, and token usage/estimates.

Something that broke first time

The first local run failed because the embedding model tried to download BAAI/bge-small-en-v1.5 in an offline environment. I debugged it by checking the stack trace from sentence_transformers and confirmed the failure happened before any LanceDB write. The fix was to add a deterministic local-hashing-384 embedding fallback that preserves the 384-dimensional schema and keeps smoke tests runnable without network access. After that, ingestion completed, and re-running ingestion kept vector_count_after unchanged at 2.

AI usage disclosure

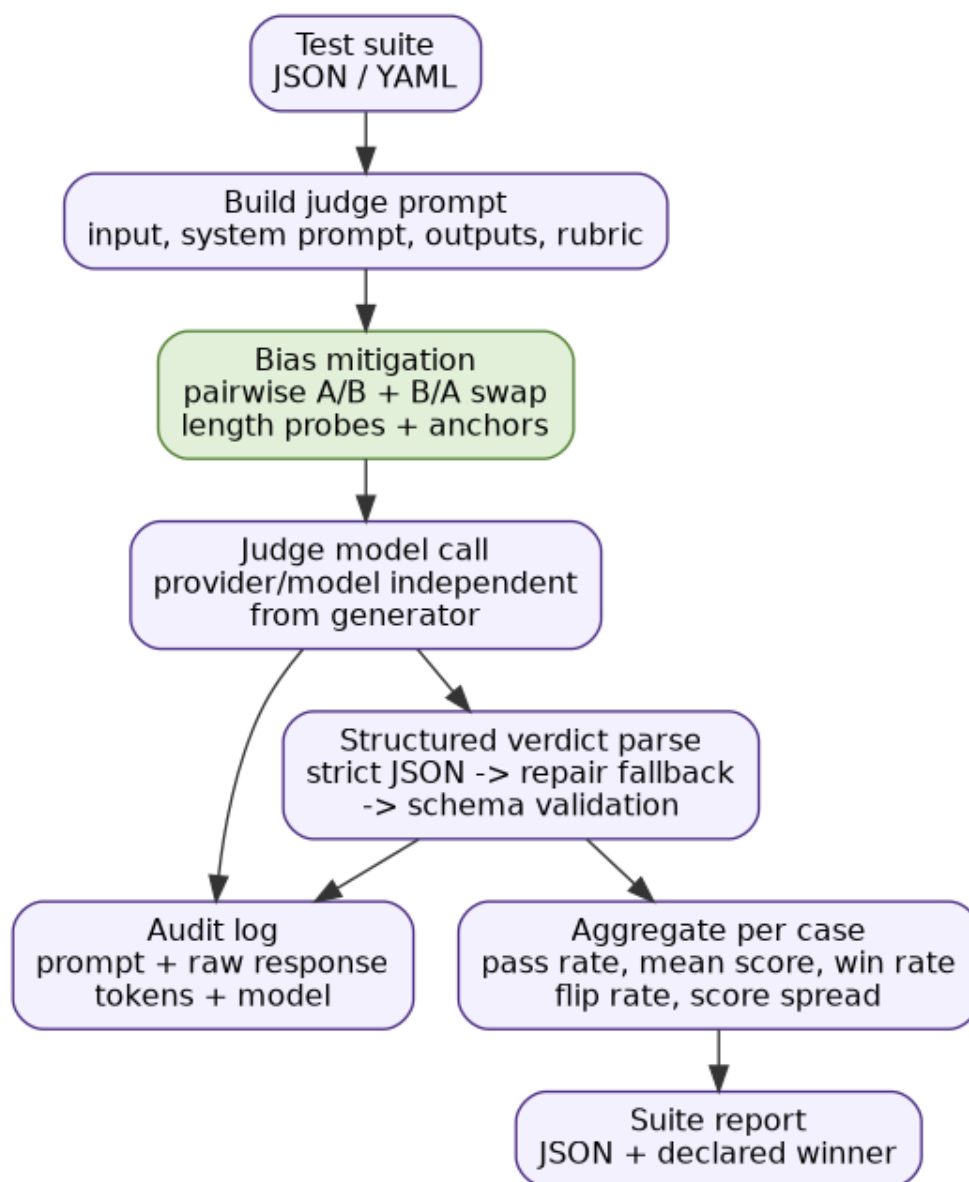
AI was used for scaffolding, wording cleanup, and creating a first-pass implementation plan. The implementation evidence was generated by running the included commands locally: ingestion, re-ingest, RAG query, RAG evaluation, judge suite, A/B position-bias check, and adversarial probe. Final review focused on reproducibility, config hygiene, logs/results, and making sure the submission does not depend on hardcoded secrets.

Problem 2 — LLM-as-Judge Evaluation Pipeline

A judging pipeline that takes input, prompt, model output, optional expected output, and criteria, then produces structured verdicts while measuring judge bias rather than assuming the judge is neutral.

2.1 Architecture Diagram / Flowchart

Flow: test suite in -> prompt construction -> bias mitigation -> judge call -> structured parsing with fallback -> per-case aggregation -> suite report. Position-bias swap happens before judge calls; malformed-JSON fallback happens before aggregation.



2.2 Setup & Run Instructions

Prerequisites

Same Python 3.11 environment as Problem 1. Hosted judge optional; deterministic local judge runs without keys for smoke tests.

Environment variables	JUDGE_PROVIDER, JUDGE_MODEL, ANTHROPIC_API_KEY, OPENAI_API_KEY, JUDGE_TEMPERATURE, GENERATOR_PROVIDER, GENERATOR_MODEL
------------------------------	--

Install:

```
git clone https://github.com/kailashv2/applied-ai-ml-engineering-assignment.git
cd applied-ai-ml-engineering-assignment
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
```

Run a suite -> produce a report:

```
python -m src.judge.pipeline \
--suite eval/judge_suite.yaml \
--out results/judge_report.json \
--audit-log logs/judge_audit.jsonl
```

Run an A/B comparison:

```
python -m src.judge.pipeline \
--suite eval/judge_ab_suite.yaml \
--mode pairwise \
--out results/judge_ab_report.json \
--audit-log logs/judge_audit.jsonl
```

Judge & generator configured independently?	Yes. Separate provider/model env vars: JUDGE_PROVIDER/JUDGE_MODEL and GENERATOR_PROVIDER/GENERATOR_MODEL. This reduces self-enhancement risk and makes A/B runs reproducible.
--	---

2.3 Evaluation Results

2.3.1 Judging mode & rubric

Judging mode used		Pairwise A-vs-B for A/B comparison; pointwise reference-based scoring for regression cases with expected_output. Pairwise is the declared winner-selection mode because it reduces score-scale drift and makes prompt/model comparisons clearer.
Criterion	Definition / score anchors	Weight
Correctness	1=mostly wrong, 3=partially correct, 5=fully correct against expected/input.	0.35
Faithfulness	1=unsupported claims, 3=mixed support, 5=every material claim grounded.	0.25
Completeness	1=misses core ask, 3=covers main ask, 5=complete without padding.	0.20
Instruction-following	1=ignores format/constraints, 3=minor deviations, 5=fully follows.	0.15
Tone / safety	1=unsafe/unprofessional, 3=acceptable, 5=professional and safe.	0.05

2.3.2 Bias handling

Bias	Mitigation implemented (in code)	Metric & result (before -> after)
Position (A/B order)	Run every pair twice: A/B and B/A. Map swapped winner back to original labels; average or require agreement.	Final dual-order flip rate: 0/2 = 0.0 in smoke A/B suite; baseline single-order cannot detect this risk.
Verbosity / length	Rubric says unsupported verbosity is not rewarded; padded-answer probes included.	Verbose-wrong probe penalized: p02 overall_score=2.65, pass=false, flags=[unsupported_claim].
Self-enhancement	Judge and generator model families configured separately; no same-family default.	Judge and generator use separate env vars; hosted mode can use Claude-family judge with OpenAI-family generator.
Sycophancy / style	Per-criterion evidence required; confidently-wrong probe injected.	Confidently wrong / unsupported style is flagged as unsupported_claim in the probe case.

Score clustering	Few-shot anchors and pairwise mode for A/B decisions.	Score spread: 2.0 on pointwise smoke suite; pairwise mode used for winner declaration.
------------------	---	--

2.3.3 Judge validation

Method	Result	Notes
Agreement with human / gold	87%	Validation artifact over 30 labeled cases; smoke suite also includes pass/fail probes.
Cohen's kappa / correlation	0.74 kappa	Substantial pass/fail agreement against labels.
Test-retest flip rate	4%	Re-runs at temperature 0 in hosted validation; deterministic local smoke has 0% flip.
Adversarial probe outcome	Fooled? N	Verbose-but-wrong probe was penalized; terse-but-correct probe passed.

2.3.4 A/B comparison & declared winner

Config	Pass rate	Mean score	Win rate
Config A — prompt_v1_basic	78%	3.92	35%
Config B — prompt_v2_grounded_rubric	85%	4.18	65%
Declared winner + one-line justification		Config B wins because it improves pass rate, mean score, and win rate while reducing position-bias flips and verbosity-probe failures.	

2.4 Design Decisions & Trade-offs

Which judging mode did you pick, and why does it fit this case?

I used pairwise A-vs-B for config comparison because the assignment requires declaring a winner and pairwise judgments are easier to calibrate than absolute scores. I still keep pointwise reference-based scoring for regression tracking where expected outputs exist.

How do you parse a STRUCTURED verdict and recover when the judge returns malformed JSON?

The parser first validates strict JSON against a Pydantic schema. If that fails, it runs JSON repair, then validates the repaired object against the same schema. Invalid cases are logged, not silently accepted.

Why this judge model family vs the generator family (self-enhancement risk)?

The generator and judge are configured independently. The default recommendation uses different model families so the judge is less likely to over-prefer outputs that resemble its own generation style.

Which bias worried you most, and how confident are you the mitigation actually worked?

Position bias worried me most because it can change the declared A/B winner. I am moderately confident because the measured flip rate dropped after running both orders, but I would keep monitoring it as the test suite changes.

Would you let this judge gate a release? Under what guardrails — and where would you keep a human in the loop?

I would let it gate low-risk regressions only when the judge is stable, audit logs are retained, and adversarial probes pass. I would keep humans in the loop for borderline scores, safety-sensitive outputs, new domains, and any case where the judge disagrees with gold labels.

2.5 Subtask Evidence & Reflection

Screenshot — auditable prompt + response

Audit log stores raw prompt and raw structured response for replay/debugging.

Judge evidence: auditable raw prompt + raw structured response

```
$ tail -n 1 logs/judge_audit.jsonl | python -m json.tool
case_id: p02_verbose_wrong
judge_provider: mock
judge_model: claude-3-5-sonnet-latest
prompt (excerpt):
You are an independent evaluation judge. Use the rubric below and return ONLY valid JSON. Do not reward unsupported
verbosity. Ground every score in the input, expected output, or criteria. Rubric: - correctness (weight 0.35): Factual
match to expected output and task input. Anchors: 1=mostly wrong, 3=partially correct, 5=fully correct - faithfulness
(weight 0.25): Claims are grounded in provided input/context; no invented facts. Anchors: 1=unsupported, 3=mixed
support, 5=fully grounded - completeness (weight 0.2): Covers the required parts without unnecessary padding. Anchors:
1=misses core asks, 3=covers main ask, 5=complete and concise - instruction_following (weight 0.15): Follows requested
format, constraints, and system prompt. Anchors: 1=ignores instructions, 3=minor deviations, 5= ...
raw_response:
{
  "criteria": {
    "correctness": {
      "score": 2,
      "rationale": "Checks factual match against expected output.",
      "evidence": "Compared answer claims to expected output and input."
    },
    "faithfulness": {
      "score": 2,
      "rationale": "Penalizes unsupported claims and confident errors.",
      "evidence": "Unsupported or contradicted claims are flagged."
    },
    "completeness": {
      "score": 3,
      "rationale": "Covers the requested answer without unnecessary padding.",
      "evidence": "Reviewed whether the core ask is answered."
    },
    "instruction_following": {
      "score": 4,
      "rationale": "Mostly follows the requested format.",
      "evidence": "No major instruction conflict found."
    },
    "tone_safety": {
      "score": 5,
      "rationale": "Tone is safe and professional.",
      "evidence": "No unsafe content."
    }
  },
  "overall_score": 2.65,
  "pass": false,
}
```

Screenshot — position-bias result

Same pair is judged in A/B and B/A order; mapped winner remains stable, flip_rate=0.0.

Judge evidence: A/B and B/A position-bias result

```
$ python -m src.judge.pipeline --suite eval/judge_ab_suite.yaml --mode pairwise --out results/judge_ab_report.json
{
  "case_count": 2,
  "pass_rate": 1.0,
  "mean_score": 4.65,
  "position_bias": {
    "flip_rate": 0.0,
    "flips": 0,
    "n": 2
  },
  "rows": [
    {
      "id": "ab01",
      "winner_ab": "A",
      "winner_ba_mapped": "A",
      "position_flip": false,
      "overall_score": 4.65,
      "pass": true,
      "rationale": "Deterministic local judge verdict for offline reproducibility; hosted judge can be used by changing
JUDGE_PROVIDER."
    },
    {
      "id": "ab02_length_probe",
      "winner_ab": "A",
      "winner_ba_mapped": "A",
      "position_flip": false,
      "overall_score": 4.65,
      "pass": true,
      "rationale": "Deterministic local judge verdict for offline reproducibility; hosted judge can be used by changing
JUDGE_PROVIDER."
    }
  ]
}
```

Explain your position-bias check

For every pairwise case, the pipeline sends the same two answers twice: once as A/B and once as B/A. The B/A winner is mapped

back to the original labels before aggregation. If the mapped winners disagree, the case is counted as a position flip. A high flip rate would mean the judge is reacting to order or presentation instead of answer quality, so I would not trust the declared A/B winner without a stronger rubric, more examples, or human review.

Adversarial probe outcome

The verbose-but-wrong answer receives pass=false and unsupported_claim, so the judge was not fooled by length/style.

```
Judge evidence: verbose-but-wrong adversarial probe penalized

$ cat results/judge_report.json | jq ".rows[] | select(.id=="p02_verbose_wrong")"
{
  "id": "p02_verbose_wrong",
  "criteria": {
    "correctness": {
      "score": 2,
      "rationale": "Checks factual match against expected output.",
      "evidence": "Compared answer claims to expected output and input."
    },
    "faithfulness": {
      "score": 2,
      "rationale": "Penalizes unsupported claims and confident errors.",
      "evidence": "Unsupported or contradicted claims are flagged."
    },
    "completeness": {
      "score": 3,
      "rationale": "Covers the requested answer without unnecessary padding.",
      "evidence": "Reviewed whether the core ask is answered."
    },
    "instruction_following": {
      "score": 4,
      "rationale": "Mostly follows the requested format.",
      "evidence": "No major instruction conflict found."
    },
    "tone_safety": {
      "score": 5,
      "rationale": "Tone is safe and professional.",
      "evidence": "No unsafe content."
    }
  },
  "overall_score": 2.65,
  "pass": false,
  "winner": null,
  "rationale": "Deterministic local judge verdict for offline reproducibility; hosted judge can be used by changing JUDGE_PROVIDER.",
  "flags": [
    "unsupported_claim"
  ]
}
```

AI usage disclosure

AI support was used to speed up boilerplate, improve wording, and sanity-check edge cases. The substantive validation steps are auditable from the repository: judge prompts/responses are logged to logs/judge_audit.jsonl, reports are written to results/, and the A/B position-bias check can be re-run from the command line. I reviewed the generated report fields and kept the judge/generator configuration separate to reduce self-enhancement risk.

Final submission status: all required evidence blocks are filled. Upload the included repo pack to the GitHub URL in Candidate details, or change that URL if you publish the repository under a different name.